

Kelvin



Projekt: Facebook Haystack a Event-Driven Architektura

Class lecturer: ALB0016

Deadline: 18.05.2026, 12:30, Time remaining: **18 hours, 10 minutes left**

Select multiple files using CTRL or SHIFT or drag them to this window

Assignment

Upload ?

Finálním završením implementace našeho cloudu bude zjednodušená implementace úložiště fotografií inspirovaná architekturou, kterou vyvinul a využíval Facebook. Ten ji detailně popisuje v článku [Finding a needle in Haystack: Facebook's photo storage \[PDF\]](#).

Vaše stávající S3 úložiště funguje, dokud do něj uživatelé nezačnou nahrávat miliony malých souborů (např. profilové fotky). Výkon tradičních souborových systémů (jako ext4 nebo NTFS) při takovém množství souborů značně degraduje. Každý soubor má svá metadata (inode). Když chcete fotku přečíst, čtecí hlava disku musí nejprve najít inode a pak fyzicky přeskočit na místo s daty. Toto je operace **disk seek** a zabírá obrovské množství času. Pokud byste do jednoho adresáře uložili třeba 1 000 000 souborů, příkaz `ls` (výpis adresáře) by trval neúnosně dlouho.

Facebook tento problém vyřešil architekturou **Haystack**. Místo ukládání milionů malých fotek vytvoří jeden obrovský soubor (např. 100 GB), kterému říká **Volume (Svazek)**. Fotky do něj pouze "připojuje" na konec (metoda *Append-only*), což je z hlediska zápisu ta nejrychlejší možná operace. Aby fotku (tzv. **Needle - Jehlu**) v tomto obřím souboru našel, drží si v rychlé relační databázi v paměti pouze tři čísla: **ID svazku**, **Offset (počáteční bajt)** a **Size (velikost)**. Čtení pak vyžaduje maximálně 1 pohyb disku.

V tomto projektu transformujete vaše dosavadní řešení na plnohodnotnou mikroslužbovou architekturu.

Poznámka k terminologii: Pojem **volume** se v diskových subsystémech používá v mnoha významech (např. technologie logických oddílů na Linuxu se nazývá Logical Volume Manager - LVM). V kontextu tohoto zadání je "Volume" jednoduše jeden velký binární soubor na disku (např. `volume_1.dat`).

Architektura systému

Váš systém se nyní bude skládat ze **4 nezávislých aplikací**, které spolu komunikují asynchronně:

1. **S3 Gateway:** Přijímá HTTP požadavky od uživatelů, řeší Pydantic validace, billing a spravuje SQLite databázi metadat (index). **Fyzicky už neukládá žádné soubory na disk!**

2. **Message Broker:** Slouží jako centrální komunikační dálnice (Pub/Sub) mezi S3 Gateway a worker nody.
3. **Image Processing Node:** Samostatná služba pro grafické úpravy obrázků přes NumPy.
4. **Haystack Node (Nová aplikace):** Fyzicky spravuje velké Volume soubory a zapisuje do nich přijatá binární data.

Úkol 1: Haystack Storage Node (Zápis a Čtení)

Vytvořte novou FastAPI aplikaci. Její hlavní zodpovědností je rychlý asynchronní zápis dat a jejich následné čtení.

1. Inicializace a Rotace svazků:

- Při startu si aplikace otevře (nebo vytvoří) binární soubor `volume_1.dat` v režimu pro přidávání ("`ab+`").
- Nebudeme implementovat "Facebook scale", nastavte si v konfiguraci maximální velikost svazku (např. 100 MB).
- *Logika rotace:* Jakmile zápis nové fotky způsobí překročení tohoto limitu, aplikace aktuální soubor uzavře a vytvoří nový (`volume_2.dat`), do kterého začne zapisovat.

2. Napojení na Message Broker:

- Aplikace se připojí jako klient (Subscriber) k vašemu Message Brokeru a začne naslouchat na tématu `storage.write`. (Pozn.: Toto naslouchání musí běžet na pozadí pomocí `asyncio.create_task`, aby nezablokovalo samotné FastAPI).

3. Zpracování zprávy (Append-only zápis):

- Když dorazí zpráva (serializovaná pomocí `MessagePack`), aplikace:
 - Zjistí aktuální pozici na konci aktivního souboru (`offset = file.tell()`).
 - Zapiše payload fotky na konec souboru (`file.write(data)`).
 - Získá velikost zapsaných dat (`size = len(data)`).
- Okamžitě odešle zpět přes brokera do tématu `storage.ack` potvrzovací zprávu ve formátu: `{"object_id": "uuid-z-gateway", "volume_id": 1, "offset": 10560, "size": 1024}`.

4. HTTP Endpoint pro čtení:

- Vytvořte GET endpoint `/volume/{volume_id}/{offset}/{size}`.
- Tento endpoint pouze otevře příslušný soubor `volume_{volume_id}.dat`, pomocí `file.seek(offset)` skočí na správné místo a vrátí přesně definovaný počet bajtů jako `StreamingResponse` nebo `Response(media_type="image/jpeg")`.

Úkol 2: Integrace S3 Gateway (Asynchronní zápis)

Upravte vaši původní S3 aplikaci (endpoint `POST /upload`), aby respektovala Event-Driven architekturu:

1. **Příjem a Odeslání:** Místo ukládání na disk vezme přijatý soubor a odešle jej přes Message Broker do tématu `storage.write`. Zpráva bude obsahovat unikátní `object_id` a samotná binární data.
2. **Eventual Consistency (Stavový automat):** Do své SQLite databáze uloží objekt, ale se statusem `status = "uploading"`. Uživatel dostane HTTP 202 Accepted. Znamená to, že systém o požadavku ví, ale ještě není fyzicky na disku.
3. **Potvrzení (ACK):** S3 Gateway musí mít na pozadí spuštěný task, který naslouchá na tématu `storage.ack`. Jakmile dorazí potvrzení od Haystack Node:
 - Vyhledá v databázi příslušné `object_id`.
 - Aktualizuje hodnoty `volume_id`, `offset` a `size`.
 - Změní status na `status = "ready"`.
 - **Až v tuto chvíli** se uživatel strhnou kredity / zaúčtuje se billing (pokud jste billing implementovali).

Úkol 3: Čtení a Soft Delete (S3 Gateway)

S3 Gateway slouží jako jediný bod kontaktu pro uživatele (API klienty). Uživatel nikdy nekomunikuje přímo s Haystack Node.

1. **Čtení (`GET /download/{object_id}`):**
 - Uživatel požádá o soubor. S3 Gateway zkontroluje databázi (a oprávnění).
 - Pokud má soubor `status == "ready"`, Gateway si přečte jeho `volume_id`, `offset` a `size`.
 - S3 Gateway interně zavolá (např. přes asynchronní knihovnu `httpx`) Haystack Node na jeho `/volume/...` endpoint.
 - Přijatá data obratem přepošle uživateli.
2. **Mazání (`DELETE /download/{object_id}`):**
 - Implementujte striktní **Soft Delete**. Z databáze v S3 Gateway objekt nezmizí, pouze se nastaví příznak `is_deleted = True`.
 - Haystack Node se o mazání **vůbec nedozví**. Data ve `volume_X.dat` fyzicky zůstávají. Disk se tak nezdržuje žádným přepisováním sektorů.

Úkol 4: Compaction (Kompakce neboli Defragmentace)

Jelikož fotky pouze přidáváme a při mazání je pouze "skryjeme" v databázi, naše Volume soubory by donekonečna rostly a obsahovaly "hluchá" (fragmentovaná) místa po smazaných souborech. Dochází tedy k tzv. fragmentaci, která není vhodná.

1. Vytvořte administrační skript `compact.py` (spouštěný např. přes cron) nebo chráněný HTTP endpoint v Haystack Node.
2. **Algoritmus kompakce:**

- Skript si pro konkrétní svazek (např. `volume_1`) vyžádá ze S3 Gateway API seznam všech **nesmazaných** objektů a jejich offsetů.
- Vytvoří nový soubor `volume_1_compacted.dat`.
- Postupně čte data ze starého souboru a přepisuje je těsně za sebe do nového souboru (čímž se zbaví děr po smazaných souborech).
- **Průběžně odesílá** na S3 Gateway nové updatované hodnoty `offset` a `volume_id` pro každý přesunutý soubor.
- Jakmile je proces hotov, starý soubor `volume_1.dat` se smaže z disku a ušetřili jste místo.

Nápověda pro práci s binárním souborem v Pythonu

Pro Haystack Node budete potřebovat nízkourovňové metody pro práci se soubory. Následuje základní ukázka:

```
# ZÁPIS (Append) - přidání na konec
with open("volume_1.dat", "ab+") as f:
    # 1. Zjistíme, kde je aktuálně konec souboru (to bude náš offset)
    offset = f.tell()

    # 2. Zápis dat (payload je typu bytes)
    f.write(payload)

    # 3. Velikost je délka zapsaných bajtů
    size = len(payload)

    print(f"Uloženo na offset {offset}, velikost {size}")

# ČTENÍ - rychlý skok na konkrétní místo
with open("volume_1.dat", "rb") as f:
    # 1. Přesun čtecí hlavy (seek) přesně na místo, kde jehla začíná
    f.seek(offset)

    # 2. Přečteme přesně požadovaný počet bajtů
    data = f.read(size)
```

Bodové ohodnocení

V rámci vývoje tohoto cloudu jste již implementovali několik úloh. První dvě úlohy budou započteny do bodování běžných cvičení / domácích úloh.

Následující blok tvoří **Projektovou část 1 (15 bodů)**: - Evoluce databáze: Alembic, Buckety a Účtování - Asynchronní komunikace: Vlastní Message Broker (Pub/Sub) - Asynchronní zpracování obrazových dat (Image Worker Node)

Zbývajících 15 bodů (Projektová část 2) můžete získat právě za implementaci tohoto zadání architektury Haystack. Neobávejte se tedy toho, že je finální projekt v systému Kelvin hodnocen pouze 15 body.